Demo - Sarvam

Yogesh Haribhau Kulkarni

Workshop Overview

What we will build: A multilingual pipeline that accepts an audio recording in any Indian language, transcribes it, translates it to English, generates an AI response using the Sarvam LLM, translates the response back to the user's language, and speaks it aloud.

What we will learn:

- Speech-to-Text with Saaras v3 (5 output modes, 23 languages)
- Text-to-Speech with Bulbul v3 (30+ voices)
- Translation (Mayura / Sarvam-Translate) and Transliteration
- Language Identification and Chat Completion
- Document Intelligence (free OCR)
- Batch processing for call analytics
- A real voice agent using LiveKit

Duration: 40 minutes. Cost: Covered by Rs. 1,000 free signup credits.

Workshop Demo Stages

#	Activity	API / Feature
1	Setup & first call	SDK install, Auth, Translate
2	Speech to Text	Saaras v3, 5 modes
3	Text to Speech	Bulbul v3, voices
4	Full STT → LLM → TTS pipeline	Saaras + Sarvam-M + Bulbul
5	Translation modes & register	Mayura, Sarvam-Translate
6	Transliteration	Script conversion
7	Language Identification	Detect language/script
8	Document Intelligence	Free OCR (PDF)
9	Batch call analytics	Batch STT + diarization + LLM
10	Voice agent (LiveKit)	Real-time voice bot

Each stage is a self-contained code block. Run them in order or jump to any stage.

Environment Setup

Pre-requisites Checklist

Before starting, verify:

- Python ≥ 3.9 installed: python □□version
- A Sarvam AI account and API key: dashboard.sarvam.ai
- Free credits received on signup (Rs. 1,000)
- (For voice agent demo) A LiveKit Cloud free account: cloud.livekit.io
- A sample audio file in any Indian language (WAV/MP3, under 30s for REST API)
- A sample PDF with Indian language text (for OCR demo)

Set environment variable:

```
export SARVAM_API_KEY="your_api_key_here"

# Windows PowerShell:
$env:SARVAM_API_KEY = "your_api_key_here"
```

Install dependencies:

```
pip install □U sarvamai python□dotenv pydub
# For voice agent demo:
pip install "livekit□agents[sarvam,openai,silero]"
```

First API Call: Translation

Verify your setup with a simple translation call:

```
import os
from sarvamai import SarvamAI

client = SarvamAI(
    api_subscription_key=os.environ["SARVAM_API_KEY"]
)

# Translate English to Hindi
response = client.text.translate(
    input="Welcome to the Sarvam AI workshop!",
    source_language_code="en□IN",
    target_language_code="hi□IN",
    speaker_gender="Female"
)
print(response)
# Expected: सर्वम एआई वर्कशॉप में आपका स्वागत है!
```

Check current credit balance: Visit dashboard.sarvam.ai/billing API Status: status.sarvam.ai

Stage 2: Speech to Text

Saaras v3 --- 23 languages, 5 output modes

Saaras v3: Basic Transcription

```
from sarvamai import SarvamAI

client = SarvamAI(api_subscription_key="YOUR_KEY")

# Mode 1: transcribe (default) □ returns text in spoken language
with open("hindi_audio.wav", "rb") as audio_file:
    response = client.speech_to_text.transcribe(
        file=audio_file,
        model="saaras:v3",
        mode="transcribe",
        language_code="hi□IN"    # or omit for auto□detect
    )
print("Transcript:", response.transcript)

# Mode 2: translate □ transcribe AND translate to English
with open("hindi_audio.wav", "rb") as audio_file:
    response = client.speech_to_text.transcribe(
        file=audio_file,
        model="saaras:v3",
        mode="translate"      # returns English text
    )
print("English:", response.transcript)
```

Supported formats: wav, mp3, aac, ogg, flac, mp4/m4a, amr Max REST duration: 30 seconds per request.

Saaras v3: All 5 Output Modes

```
modes = ["transcribe", "translate", "verbatim",
         "translit", "codemix"]

for mode in modes:
    with open("hindi_audio.wav", "rb") as audio_file:
        response = client.speech_to_text.transcribe(
            file=audio_file,
            model="saaras:v3",
            mode=mode
        )
    print(f"[{mode}] {response.transcript}")

# Expected output for "Main office ja raha hoon":
# [transcribe] मैं ऑफिस जा रहा हूँ
```

```
# [translate]   I am going to the office
# [verbatim]    मैं ऑफिस जा रहा हूँ (word for word)
# [translit]    Main office ja raha hoon
# [codemix]     Main office जा रहा हूँ
```

When to use each mode:

transcribe	Call centre transcripts
translate	Multilingual dashboards needing English output
verbatim	Legal / medical records (no normalisation)
translit	SMS, Roman-script input systems
codemix	Social media, youth audience apps

Real-Time STT via WebSocket

For live voice applications (voice bots, live captioning), use the WebSocket API:

```
import asyncio
from sarvamai import AsyncSarvamAI

async def stream_transcription():
    client =
     AsyncSarvamAI(api_subscription_key="YOUR_KEY")

    async with client.speech_to_text.stream(
        model="saaras:v3",
        mode="transcribe",
        language_code="hi-IN"
    ) as stream:
        # Push audio chunks (16kHz, PCM 16-bit WAV)
        with open("live_audio.wav", "rb") as f:
            while chunk := f.read(4096):
                await stream.send_audio(chunk)

        await stream.end_stream()

        # Receive transcripts as they arrive
        async for event in stream:
            print(event.transcript)

asyncio.run(stream_transcription())
```

WebSocket constraint: Only wav and raw PCM formats (pcm_s16le, pcm_l16, pcm_raw) supported for streaming. Sample rate: 16kHz.

Stage 3: Text to Speech

Bulbul v3 --- 30+ voices, 11 languages

Bulbul v3: Generate Speech

```
from sarvamai import SarvamAI

client = SarvamAI(api_subscription_key="YOUR_KEY")

# Generate Hindi speech with a professional male voice
response = client.text_to_speech.convert(
    target_language_code="hi-IN",
    text="नमस्ते! आपका EMI भुगतान 15 जनवरी को देय है।",
    model="bulbul:v3",
    speaker="anand",            # professional Hindi male
    speech_sample_rate=16000, # 8000 / 16000 / 24000
    enable_preprocessing=True # handle abbreviations,
     numbers
)

# Save to WAV file
with open("output_hindi.wav", "wb") as f:
    f.write(bytes(response.audios[0]))
print("Audio saved to output_hindi.wav")
```

Try different speakers for the same text:

```
for speaker in ["anand", "rahul", "priya", "simran"]:
    resp = client.text_to_speech.convert(
        target_language_code="hi-IN",
        text="नमस्ते, मैं आपकी कैसे मदद कर सकता हूँ?",
        model="bulbul:v3", speaker=speaker)
    with open(f"voice_{speaker}.wav", "wb") as f:
        f.write(bytes(resp.audios[0]))
```

Stage 4: Full Pipeline

STT → LLM → TTS in any Indian language

End-to-End: Voice In, Voice Out

```
from sarvamai import SarvamAI
import os

client =
    SarvamAI(api_subscription_key=os.environ["SARVAM_API_KEY"])

# Step 1: Transcribe user's speech (any Indian language)
with open("user_query.wav", "rb") as f:
    stt = client.speech_to_text.transcribe(
        file=f, model="saaras:v3", mode="transcribe"
    )
user_text = stt.transcript
detected_lang = stt.language_code    # auto-detected language
print(f"[STT] ({detected_lang}): {user_text}")

# Step 2: Get LLM response (sarvam-m is FREE)
llm_resp = client.chat.completions(
    messages=[
        {"role": "system", "content":
         f"You are a helpful assistant. Reply in
          {detected_lang}."},
        {"role": "user", "content": user_text}
    ],
    model="sarvam-m"  # free model
)
reply_text = llm_resp.choices[0].message.content
print(f"[LLM]: {reply_text}")

# Step 3: Speak the response in the user's language
tts = client.text_to_speech.convert(
    target_language_code=detected_lang,
    text=reply_text, model="bulbul:v3", speaker="priya"
)
with open("agent_response.wav", "wb") as f:
    f.write(bytes(tts.audios[0]))
print("Response audio saved!")
```

Stage 5 & 6: Translation and Transliteration

Translation: Register Modes

```
text = "Your account has been blocked due to non-payment."

for mode in ["formal", "colloquial", "modern-colloquial"]:
    resp = client.text.translate(
        input=text,
        source_language_code="en-IN",
        target_language_code="hi-IN",
        mode=mode
    )
    print(f"[{mode}]: {resp.translated_text}")

# formal:           भुगतान" न होने के कारण आपका खाता
    बंद कर दिया गया है।"
# modern-colloquial: "Non-payment की वजह से आपका
    account block हो गया है।"
# colloquial:       पैसे" नहीं भरे तो account band हो गया।"
```

Bulk translation with Sarvam-Translate (all 23 languages):

```
# For languages beyond the 11 supported by Mayura,
# use the sarvam-translate endpoint:
response = client.text.translate(
    input="मेरा नाम विनायक है।",
    source_language_code="hi-IN",
    target_language_code="sat-IN",   # Santali
    model="sarvam-translate:v1"
)
print(response.translated_text)
```

Transliteration and Language Identification

Transliteration --- convert script, preserve pronunciation:

```
# Devanagari -> Roman (romanisation)
response = client.text.transliterate(
    input="नमस्ते, मेरा नाम विनायक है।",
    source_language_code="hi-IN",
    target_language_code="en-IN",
    spoken_form=True,
    numerals_format="international"
)
print(response.transliterated_text)
# Output: "namaste, mera naam vinayak hai."

# Roman -> Devanagari (for users typing in Roman)
response = client.text.transliterate(
    input="Main office ja raha hoon",
    source_language_code="en-IN",
    target_language_code="hi-IN"
)
print(response.transliterated_text)
# Output: मैं" ऑफिस जा रहा हूँ"
```

Language Identification:

```
resp = client.text.identify_language(
    input="नमस्कार, तुम्ही कसे आहात?"
)
print(resp.language_code)   # "mr-IN"  (Telugu)
print(resp.script_code)     # "Deva"
```

Stage 8: Document Intelligence

Free OCR for Indian language PDFs

Document OCR: Extract Text from Indian PDFs

Currently free for all plans (max 10 pages per request):

```
import base64, httpx, os

SARVAM_KEY = os.environ["SARVAM_API_KEY"]

# Read and encode the PDF
with open("hindi_document.pdf", "rb") as f:
    pdf_b64 = base64.b64encode(f.read()).decode()

response = httpx.post(
    "https://api.sarvam.ai/v1/parse",
    headers={"api-subscription-key": SARVAM_KEY},
    json={"file": pdf_b64, "file_type": "pdf"}
)

data = response.json()
print("Extracted text:\n", data["text"])
# Structured page-by-page output:
for i, page in enumerate(data.get("pages", []), 1):
    print(f"\n--- Page {i} ---")
    print(page["text"])
```

Use this for:

- Government forms and certificates in Hindi/Gujarati/Marathi
- Bank statements, invoices in regional scripts
- Land records, academic mark sheets
- Rate limit: 10 req/min uniform across all plans

Document OCR → Translate → TTS Pipeline

Combine Document Intelligence with Translation and TTS:

```
import base64, httpx, os
from sarvamai import SarvamAI

client =
    SarvamAI(api_subscription_key=os.environ["SARVAM_API_KEY"])

# Step 1: OCR the PDF
with open("gujarati_document.pdf", "rb") as f:
    pdf_b64 = base64.b64encode(f.read()).decode()

ocr = httpx.post(
    "https://api.sarvam.ai/v1/parse",
    headers={"api-subscription-key":
      os.environ["SARVAM_API_KEY"]},
    json={"file": pdf_b64, "file_type": "pdf"}
).json()

extracted_text = ocr["text"][:2000]  # first 2000 chars
print("Extracted:", extracted_text[:200], "...")

# Step 2: Translate to English
translation = client.text.translate(
    input=extracted_text,
    source_language_code="gu-IN",
    target_language_code="en-IN"
)
print("English:", translation.translated_text[:200])

# Step 3: Read it aloud in Gujarati
tts = client.text_to_speech.convert(
    target_language_code="gu-IN",
    text=extracted_text[:500],
    model="bulbul:v3", speaker="priya"
)
with open("document_audio.wav", "wb") as f:
    f.write(bytes(tts.audios[0]))
print("Document audio saved.")
```

Stage 9: Batch Call Analytics

Transcribe, diarise, and analyse support calls

Batch STT with Diarization

```
from sarvamai import SarvamAI
from pathlib import Path

client = SarvamAI(api_subscription_key="YOUR_KEY")

# Create batch job: up to 20 files, up to 1 hour each
job = client.speech_to_text_translate_job.create_job(
    model="saaras:v3",
    mode="translate",          # translate to English
    with_diarization=True,   # speaker labels
)

# Upload audio files (call recordings)
job.upload_files(
    file_paths=["call1.mp3", "call2.mp3"],
    timeout=300   # increase for large files
)
job.start()
print("Job started:", job.job_id)

job.wait_until_complete()

if job.is_failed():
    print("Job failed!")
else:
    # Download JSON transcription outputs
```

```
    output_dir = Path("transcriptions")
    output_dir.mkdir(exist_ok=True)
    job.download_outputs(output_dir=str(output_dir))
    print("Transcriptions saved to:", output_dir)
```

Parse Diarized Output and Analyse with LLM

```
import json
from pathlib import Path
from sarvamai import SarvamAI

client = SarvamAI(api_subscription_key="YOUR_KEY")

# Parse the diarized JSON output
for json_file in Path("transcriptions").glob("*.json"):
    with open(json_file) as f:
        data = json.load(f)

    entries = data.get("diarized_transcript", {}).get("entries", [])
    conversation = "\n".join(
        f"{e['speaker_id']}: {e['transcript']}"
        for e in entries
    )
    print(conversation[:300])
```

```
    # Send to LLM for structured analysis (sarvam-m is
      free)
    resp = client.chat.completions(
        messages=[
            {"role": "system", "content":
             "You are a call analytics expert."},
            {"role": "user", "content":
             f"Analyse this call:\n{conversation}\n\n"
             "Identify: 1) Which speaker is agent vs
      customer? "
             "2) Was the issue resolved? 3) Customer
      sentiment?"}
        ],
        model="sarvam-m"   # free!
    )
    print("\nAnalysis:", resp.choices[0].message.content)
```

Use cases: BPO quality monitoring, banking call compliance, e-commerce support analytics.

Stage 10: Real-Time Voice Agent

LiveKit + Sarvam AI

Build a Multilingual Voice Agent with LiveKit

Install: pip install "livekit-agents[sarvam,openai,silero]" python-dotenv

Create .env file:

```
LIVEKIT_URL=wss://your-project.livekit.cloud
LIVEKIT_API_KEY=APIxxxxxxxx
LIVEKIT_API_SECRET=xxxxxxxxxxxx
SARVAM_API_KEY=sk_xxxxxxxx
OPENAI_API_KEY=sk-proj-xxxxxxxx   # for GPT-4o LLM
```

Create tutor_agent.py:

```
from dotenv import load_dotenv
from livekit.agents import JobContext, WorkerOptions, cli
from livekit.agents.voice import Agent, AgentSession
from livekit.plugins import openai, sarvam

load_dotenv()

class TutorAgent(Agent):
    def __init__(self):
        super().__init__(
            instructions="""You are a friendly tutor.
            Explain concepts clearly in the student's
      language.
            Support Hindi, Gujarati, Telugu, Bengali,
      Marathi.""",
            stt=sarvam.STT(
                language="unknown",      # auto-detect
                model="saaras:v3",
                mode="transcribe"
            ),
            llm=openai.LLM(model="gpt-4o"),
            tts=sarvam.TTS(
                target_language_code="hi-IN",
                model="bulbul:v3",
                speaker="anand"
            ),
        )
    async def on_enter(self):
        self.session.generate_reply()

async def entrypoint(ctx: JobContext):
    session = AgentSession()
    await session.start(agent=TutorAgent(), room=ctx.room)

if __name__ == "__main__":
    cli.run_app(WorkerOptions(entrypoint_fnc=entrypoint))
```

Run and Test the Voice Agent

```
# Terminal 1: Start the agent worker
python tutor_agent.py dev

# Terminal 2: Test in console mode (type instead of speak)
python tutor_agent.py console

# Try multilingual inputs:
# > गणित" में भिन्न क्या होता है?"          (Hindi)
# > भाग" म्हणजे काय??"                    (Marathi)
# > "What is Pythagoras theorem?"          (English)
```

Other pre-built voice agent examples in the Sarvam Cookbook:

Agent	Use Case
Collection Agent	EMI payment reminders in 11 languages
Government Scheme Agent	Explain PM schemes to rural users
Tutor Agent	Multilingual educational assistant
Loan Advisory Agent	Vernacular loan guidance chatbot

Docs: docs.sarvam.ai/api-reference-docs/cookbook/example-voice-agents

Workshop Summary

What We Built

In 40 minutes, using primarily free tier:

- translate_demo.py: English ↔ 11 Indian languages with register control
- stt_modes_demo.py: Audio transcription in 5 modes (transcribe/translate/verbatim/translit/codemix)
- tts_voices_demo.py: Speech synthesis with 30+ voices in 11 languages
- pipeline.py: Full voice pipeline --- speak in Hindi, get reply in Hindi
- transliterate_demo.py: Roman ↔ Devanagari/Gujarati/Telugu etc.
- ocr_pipeline.py: Extract text from Indian language PDF → translate → speak
- call_analytics.py: Batch transcription, diarization, LLM analysis of calls
- tutor_agent.py: Live multilingual voice agent via LiveKit

What is completely free to run:

- Sarvam-M chat completion --- no cost, no usage limit beyond rate limits
- Document Intelligence OCR --- currently free for all plans
- Everything else runs on Rs. 1,000 free signup credits

Sarvam AI API Cheat Sheet

Task	SDK Call	Notes
STT transcribe	speech_to_text.transcribe()	mode=transcribe
STT translate	speech_to_text.transcribe()	mode=translate
STT codemix	speech_to_text.transcribe()	mode=codemix
STT streaming	speech_to_text.stream()	WebSocket, WAV only
Batch STT	speech_to_text_translate_job	Up to 1hr/file
TTS generate	text_to_speech.convert()	30+ voices
TTS stream	text_to_speech.stream()	WebSocket
Translate	text.translate()	11-23 langs
Transliterate	text.transliterate()	Script only
Detect language	text.identify_language()	lang + script
Chat completion	chat.completions()	sarvam-m = free
Document OCR	POST /v1/parse	currently free

Auth header: api-subscription-key: YOUR_KEY Base URL: https ://api.sarvam.ai/v1/ SDK: pip install sarvamai

Key Limits and Tips

Rate Limits (Starter / free plan):

API	Provisioned (Starter)
STT REST	60 req/min
STT WebSocket	20 concurrent
Batch STT	20 req/min
TTS REST	60 req/min (Bulbul v3: 30)
Translation	60 req/min
Chat (sarvam-m)	60 req/min
Chat (30B/105B)	40 req/min
Document OCR	10 req/min (all plans)

Key tips:

- STT REST max: 30 seconds per request. Use Batch API for longer audio.
- Transliteration API max: 1000 characters per request. Split long texts.
- Transliteration between two Indic scripts (e.g., Hindi → Bengali) is not supported.
- WebSocket streaming STT only accepts WAV/PCM at 16kHz.
- For Batch API status polling, add a 5ms minimum delay between polls.
- Use language_code="unknown" to auto-detect language in STT.
- Document OCR: max 10 pages per request.

Exercises for Self-Study

- Exercise 1: Build a Python script that accepts a WAV file from the command line, auto-detects the language, transcribes it, and saves both the transcript and a translation to English.
- Exercise 2: Create a multilingual FAQ bot: user asks a question in any Indian language (via text input), translate to English, answer with Sarvam-M LLM (free), translate answer back, and speak it with Bulbul v3.
- Exercise 3: Process a Kannada PDF (e.g., a government circular). Extract text, translate each paragraph to English, and save both versions side by side.
- Exercise 4: Build a simple call quality tool: upload 3 sample MP3 recordings, run Batch STT with diarization, and print a summary table (speaker talk-time, sentiment, issue type) using the free Sarvam-M LLM.
- Exercise 5: Extend the LiveKit voice agent to a Government Scheme Agent that explains one state govt. scheme in the language the user speaks.
- Exercise 6: Experiment with numerals_format and spoken_form in transliteration. Generate TTS for ``Your account balance is Rs. 5,32,450" in three different languages and compare.

Resources

- Dashboard: dashboard.sarvam.ai --- API keys, credits, usage analytics
- Docs: docs.sarvam.ai
- API Reference: docs.sarvam.ai/api-reference-docs/introduction
- Models: docs.sarvam.ai/.../models
- Pricing: docs.sarvam.ai/.../pricing
- GitHub Cookbook: github.com/sarvamai/sarvam-ai-cookbook
- Discord: discord.gg/5rAsykttcs --- community support
- API Status: status.sarvam.ai
- Pipecat Sarvam STT: docs.pipecat.ai/server/services/stt/sarvam
- Contact: developer@sarvam.ai

Workshop complete. Build for a billion.[0.5em] Happy coding!